

# **CAN Data logger using Arduino Nano and microSD card for BMS Diagnostics**

PRATYUSH MESHAM  
IIT(BHU)

JUNE 2025

## CERTIFICATE

---

This is to certify that the project entitled **“CAN Data logger using Arduino Nano and microSD card for BMS Diagnostics”** submitted by Pratyush Meshram of Indian Institute of Technology (Banaras Hindu University), Varanasi, was carried out as part of the internship at **Inverted Energy Pvt. Ltd.**, is an original and authentic work. The project was undertaken as part of practical training and professional exposure in the domain of electronics design and embedded systems.

The work was completed under the guidance and supervision of the technical team at Inverted Energy Pvt. Ltd., and demonstrates the application of theoretical knowledge to real-world industrial needs, such as detecting reverse polarity in battery packs and communicating the status via CAN protocol.



Mr. Akash Tiwari  
Manager (Electronics and Electrical Team)  
Inverted Energy Pvt. Ltd.

## DECLARATION

---

I hereby declare that the work presented in this project titled **“CAN Data logger using Arduino Nano and microSD card for BMS Diagnostics”** is an original and authentic record of the work carried out by me during my internship at **Inverted Energy Pvt. Ltd.** This work was done as part of my academic engagement as a student of Electronics Engineering, Indian Institute of Technology (Banaras Hindu University), Varanasi. It does not contain any part of the work, which has been submitted for the award of any degree either in this Institute or in other University/Deemed University without proper citation.



Pratyush Meshram

Indian Institute of Technology (Banaras Hindu University)

## Contents

| TOPICS                  | PAGE NO. |
|-------------------------|----------|
| Abstract                | 4        |
| Components Used         | 5        |
| Schematic Diagram       | 5        |
| PCB Layout              | 6        |
| Working Principle       | 7        |
| Arduino Code Summary    | 8        |
| Results and Conclusions | 10       |
| Applications            | 10       |
| Future Scope            | 10       |
| Appendix                | 11       |
| Bibliography            | 11       |

## Abstract

---

**During battery management system (BMS) development and diagnostics, capturing CAN bus communication is essential for observing internal parameters such as voltage, current, temperature, state of charge, and fault codes. Traditionally, this logging process relies on PC-based setups, involving bulky equipment like laptops and specialized interfaces such as PEAK CAN (PCAN) adapters. These tools not only add cost and complexity but also hinder portability and on-site flexibility, especially in automotive or field environments.**

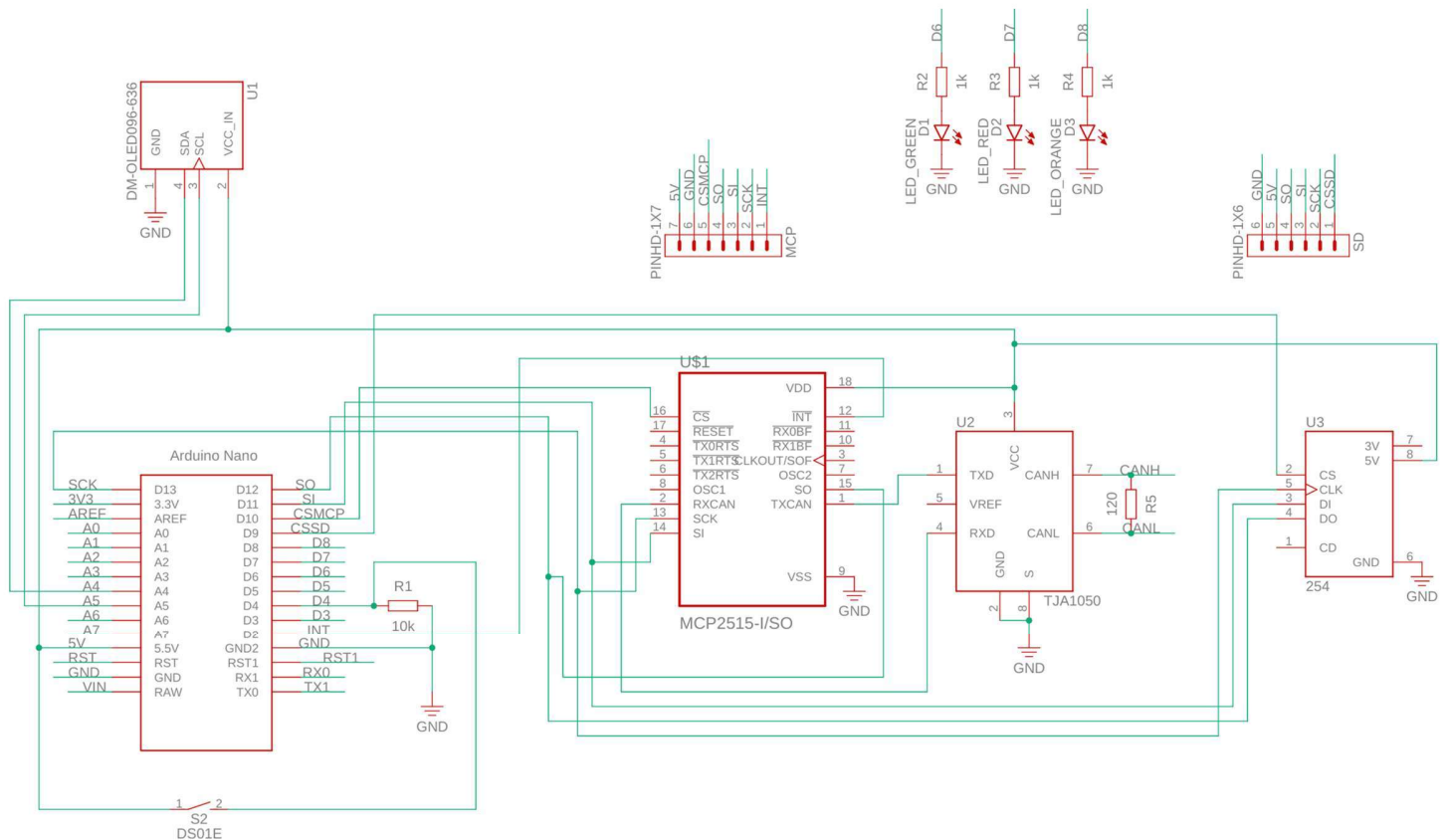
**To address this challenge, I present a compact, standalone CAN logger circuit that eliminates the need for both a laptop and PCAN interface. This device allows engineers to monitor and record BMS parameters by simply connecting CAN\_H and CAN\_L lines directly to the logger and flipping a toggle switch. The system logs all incoming CAN frames onto a microSD card, enabling efficient offline analysis without requiring a computer during data acquisition.**

**By minimizing hardware dependencies and maximizing portability, this CAN logger offers a cost-effective and field-friendly alternative for embedded diagnostics and BMS testing. Its plug-and-play nature ensures that even in constrained environments, engineers can reliably capture critical data without compromising workflow.**

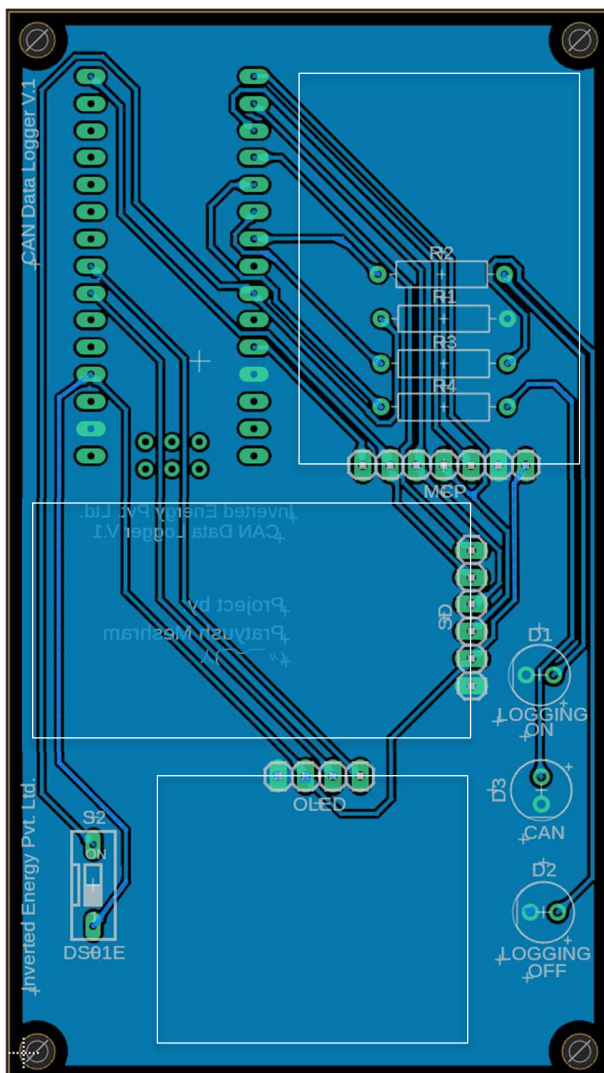
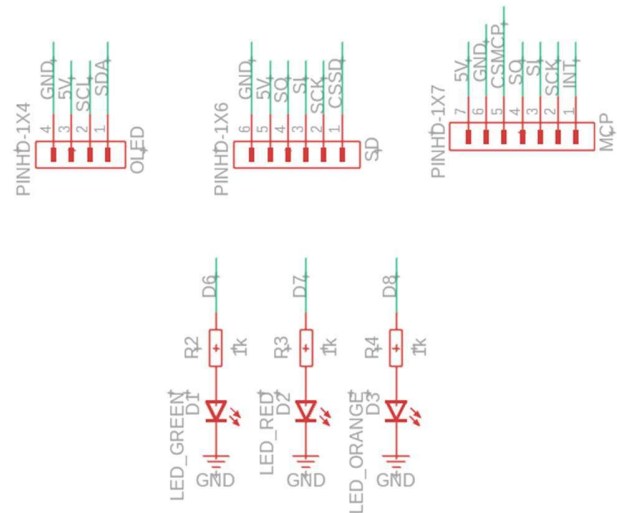
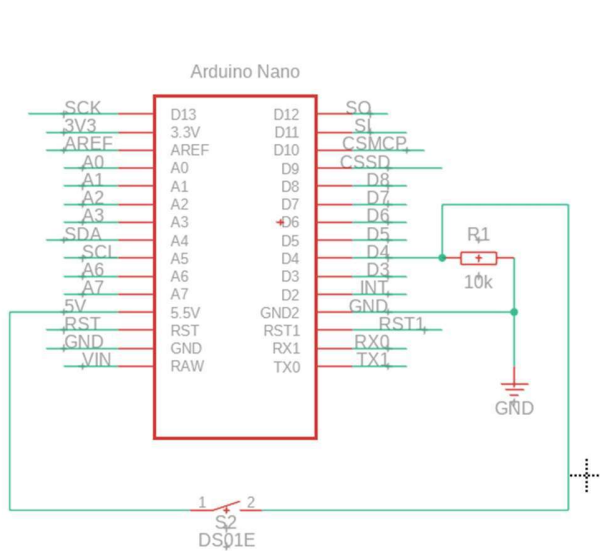
# Components Used

- Arduino Nano (ATmega328P)
- MCP2515 CAN Controller Module
- TJA1050 CAN Transceiver
- MicroSD card reader module
- OLED display
- Switch (To toggle Logging)
- LEDs (Logging Started (Green), Logging Stopped (Red), CAN trace detector (Orange))
- Resistors: 120Ω, 1kΩ, 10kΩ
- 9V Battery (for external power to Arduino Nano)

# Schematic Diagram



# PCB Layout



The PCB layout for the CAN Logger was designed with a focus on modularity, reliability, and ease of assembly. All traces were set to a width of **12 mil**, providing sufficient current capacity for 5V signals while maintaining a clean and compact routing profile. Teardrops were added at all pad and via junctions to enhance mechanical durability and reduce the risk of trace damage during soldering. Due to the lack of available footprints in Fusion 360 for the SD card module, MCP2515 CAN transceiver, and OLED display, male header footprints were used instead, with ample space left around each to accommodate the physical breakout modules. These modules can be directly soldered onto the headers, ensuring full electrical compatibility and simplifying integration. This modular approach allows for easy prototyping, replacement, and future upgrades, while maintaining signal integrity through short, direct routing of critical lines like SPI and CAN. The board layout also includes clearly labelled headers and LED indicators to assist with debugging and user interaction. Care was taken to avoid trace congestion and ensure logical placement of components for smooth assembly. Overall, the PCB is optimized for field deployment, offering a robust and adaptable platform for standalone CAN logging.

# Working Principle

---

- **Power Supply & Initialization**
  - The system is powered via a 5V source (e.g., USB or battery through voltage regulation).
  - On startup, the Arduino initializes the OLED display, SD card interface, and MCP2515 CAN controller.
  - LED indicators (green, red, orange) reflect system status: logging active, inactive, and CAN activity.
- **CAN Bus Communication**
  - The MCP2515 module interfaces with the CAN bus using SPI communication with the Arduino.
  - CAN\_H and CAN\_L lines are connected directly to the BMS or any CAN-enabled device.
- A **120Ω termination resistor** is placed across CAN\_H and CAN\_L to match the characteristic impedance of the CAN bus and prevent signal reflections. This resistor is essential when the logger is at one end of the bus or used in a standalone setup.
- **Switch-Controlled Logging**
  - A physical switch connected to digital pin D4 is used to control logging.
  - When the switch is turned ON, the system begins monitoring CAN traffic.
  - Logging only starts when valid CAN frames are received, ensuring that empty logs are avoided.
- **Data Acquisition & Storage**
  - Incoming CAN frames are read by the MCP2515 and passed to the Arduino.
  - Each frame includes an identifier (CAN ID) and up to 8 bytes of data.
  - The Arduino formats this data into a readable string and logs it to a uniquely named .txt file on the microSD card.
  - Logging continues until the switch is turned OFF or power is removed.
- **OLED Display Feedback**
  - The OLED provides real-time feedback such as “Logging STARTED,” “Logging STOPPED,” or “No CAN data transmitting...” based on system state and CAN activity.
  - CAN data is not shown on the OLED to preserve memory and display clarity; it is only printed to the Serial Monitor.
- **System Behaviour on SD Removal**
  - If the SD card is removed during operation, the system detects it and pauses logging.
  - Upon reinsertion, the SD card is automatically reinitialized, and logging resumes without requiring a reset.



# Arduino Code Summary

The Arduino code for the CAN Logger is structured to efficiently manage CAN communication, SD card logging, and user feedback through LEDs and an OLED display. It uses optimized libraries such as **SSD1306Ascii** for memory-efficient OLED control and **mcp2515.h** for interfacing with the CAN controller via SPI. The **SD.h** and **SPI.h** libraries handle file operations and peripheral communication.

## Key Functional Blocks:

- **Peripheral Initialization (setup()):**

- Initializes the OLED display, SD card, and MCP2515 CAN controller.
- Sets the CAN bitrate to 500 kbps and configures normal operating mode.

```
void setup() {
  Serial.begin(9600);
  while (!Serial);
  Serial.println(F("=== CAN Logger Initializing ==="));

  Wire.begin();
  oled.begin(&Adafruit128x64, 0x3C);
  oled.setFont(System5x7);
  oled.clear();
  oled.setCursor(0, 0);
  oled.println(F("Initializing..."));
}
```

- **Switch-Controlled Logging:**

- A physical switch connected to pin D4 controls whether logging is enabled.
- Logging only starts when the switch is ON and CAN data is received.

```
if (switchState == HIGH && !loggingEnabled) {
  loggingEnabled = true;
  justEnabled = true;
  fileIndex++;
  currentFilename = "log_" + String(fileIndex) + ".txt";
  Serial.print(F("Logging enabled, new file: "));
  Serial.println(currentFilename);
}
```

- **CAN Frame Reading and SD Logging:**

- The MCP2515 reads incoming CAN frames, which are formatted and saved to a uniquely named .txt file on the microSD card.

```
if (CAN.readMessage(&canMsg) == MCP2515::ERROR_OK) {
  digitalWrite(ORANGE_LED, HIGH);
  lastCANTime = millis();
  noDataDisplayed = false;
}
```

- **OLED Display Feedback:**

- Displays system status such as “Logging STARTED,” “Logging STOPPED,” or “No CAN data transmitting...” based on switch state and CAN activity.
- CAN data is not shown on the OLED to conserve memory.

```
Serial.println(F("Received CAN:"));
Serial.println(canData);

if (loggingEnabled && currentFilename != "" && sdAvailable) {
  if (justEnabled) {
    Serial.println(F("Starting to log CAN data to SD...\n"));
    justEnabled = false;
  }

  dataFile = SD.open(currentFilename, FILE_WRITE);
  if (dataFile) {
    dataFile.println(canData);
    dataFile.close();
    Serial.println(F("Data logged.\n"));
  } else {
    Serial.println(F("Error writing to SD\n"));
  }
}
```

- **SD Card Reinsertion Handling:**

- The code checks if the SD card is removed or reinserted and reinitializes it automatically.
- This ensures logging resumes without needing to reset the Arduino.

```
void loop() {
  // Check SD card status
  if (!SD.begin(SD_CS_PIN)) {
    if (sdAvailable) {
      Serial.println(F("SD card removed.));
      sdAvailable = false;
    }
  } else {
    if (!sdAvailable) {
      Serial.println(F("SD card reinserted. Reinitializing..."));
      sdAvailable = true;

      oled.clear();
      oled.setCursor(0, 0);
      oled.println(F("SD card detected"));
      oled.setCursor(0, 1);
      oled.println(F("Ready to log"));
      delay(1000);
    }
  }
}
```

## Results and Conclusions

---

- The CAN Logger successfully recorded BMS CAN data without needing a laptop or PCAN interface.
- Logging begins automatically when valid CAN frames are received and the switch at pin **D4 is turned ON**.
- Data is saved to a uniquely named .txt file on the microSD card.
- **OLED display provides real-time feedback:**
  - “Logging STARTED” when logging begins
  - “Logging STOPPED” when disabled
  - “No CAN data transmitting...” during inactivity
- **LED Indicators:**
  - Green LED (Pin 6): ON when logging is active
  - Red LED (Pin 7): ON when logging is disabled
  - Orange LED (Pin 8): ON when CAN frames are received; OFF after 1 second of inactivity
- SD card reinsertion is detected automatically, logging resumes without reset.
- Modular PCB design with male headers allows direct soldering of breakout modules.
- Overall, the system is portable, reliable, and ideal for field diagnostics and embedded testing.

## Applications

---

- **Portable BMS diagnostics** without the need for a laptop or PCAN interface.
- **Field data logging** of CAN bus communication in automotive and industrial systems.
- **Standalone monitoring tool** for capturing and analyzing real-time CAN parameters.

## Future Scope

---

- **Query-based CAN communication** can be implemented to actively request specific BMS parameters (e.g., voltage, temperature, SOC) rather than passively logging all frames, enabling targeted diagnostics and reduced data clutter.
- **Automatic CAN bitrate detection** to eliminate the need for manual code updates when switching between systems with different CAN speeds (e.g., 125 kbps, 250 kbps, 500 kbps), making the logger plug-and-play across platforms.
- **Wireless data transmission** via Bluetooth or Wi-Fi for real-time streaming and remote access to logged data.

## Appendix

### Bill of Materials (BoM)

| Category            | Component Description                                     | Package | Quantity |
|---------------------|-----------------------------------------------------------|---------|----------|
| Microcontroller     | Arduino NANO (2.54mm pitch)                               |         | 1        |
| CAN Module          | MCP2515 CAN Bus Module with TJA1050 Transceiver           |         | 1        |
| MicroSD Card Module | Micro SD Card Reader Module                               |         | 1        |
| OLED display        | GoldenMorning 0.96 Inch I2C/IIC 4-Pin OLED Display Module |         | 1        |
| LED Indicators      | 5mm Red LED                                               | THT     | 1        |
| LED Indicators      | 5mm Orange LED                                            | THT     | 1        |
| LED Indicators      | 5mm Green LED                                             | THT     | 1        |
| Resistors           | 1k $\Omega$ (0.25 W)                                      | THT     | 3        |
| Resistors           | 10k $\Omega$ (0.25 W)                                     | THT     | 1        |
| Resistors           | 120 $\Omega$ (0.25 W)                                     | THT     | 1        |
| Connectors          | 4-pin headers (2.54mm pitch)                              | THT     | 1        |
| Connectors          | 6-pin headers (2.54mm pitch)                              | THT     | 1        |
| Connectors          | 7-pin headers (2.54mm pitch)                              | THT     | 1        |
| Switch              | DS01E                                                     | THT     | 1        |

Bill of Materials (BoM) for the final PCB layout after successfully testing the circuit on the breadboard.

## Bibliography

- <https://circuitdigest.com/microcontroller-projects/arduino-can-tutorial-interfacing-mcp2515-can-bus-module-with-arduino>
- <https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial>
- <https://copperhilltech.com/blog/can-bus-prototyping-with-arduino-uno-a-simple-can-bus-network-monitoring-and-diagnostics-program/>
- <https://www.instructables.com/CAN-Bus-Sniffing-and-Broadcasting-with-Arduino/>
- <https://students.optimumg.com/wp-content/uploads/2017/04/vehicledatacan.pdf>
- <https://www.orionbms.com/general/retrieving-data-obd2-canbus/>